Experiment 6

Aim: Write an API endpoint that returns a 404 Not Found error for invalid routes.

MyWebsite/urls.py

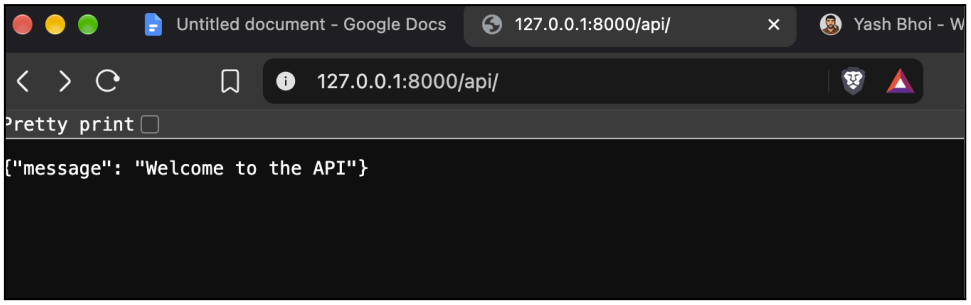
```
from django.urls import path
from django.http import JsonResponse
def home(request):
return JsonResponse({'message': 'Welcome to the API'})
def custom_404(request, exception):
return JsonResponse(
{'error': '404 Not Found', 'message': 'The requested route does not exist'},
status=404,
)
handler404 = custom_404
urlpatterns = [
path('api/', home),
]
```

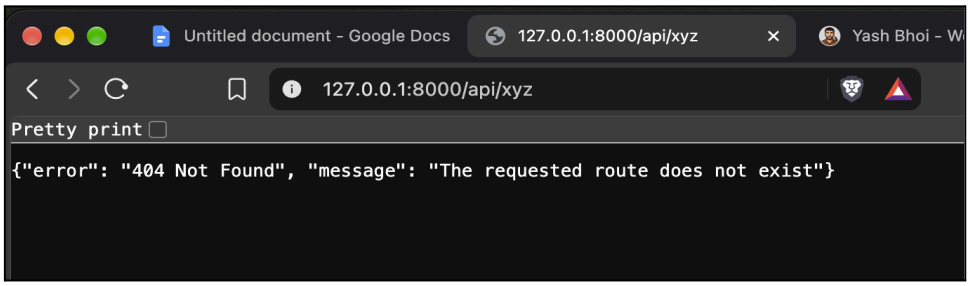
MyWebsite/settings.py (the custom 404 handler works only when DEBUG is off)

```
DEBUG = False
ALLOWED_HOSTS = ['*']
```

Output

```
$ curl -i http://127.0.0.1:8000/api/
HTTP/1.1 200 OK
Content-Type: application/json
{"message": "Welcome to the API"}
$ curl -i http://127.0.0.1:8000/api/xyz
HTTP/1.1 404 Not Found
Content-Type: application/json
{"error": "404 Not Found", "message": "The requested route does not exist"}
```





Experiment 7

Aim: Create a new Django project named MyWebsite and run the development server.

Commands

```
pip install django
django-admin startproject MyWebsite
cd MyWebsite
python manage.py runserver
```

Project structure

```
MyWebsite/
manage.py
MyWebsite/
__init__.py
settings.py
urls.py
asgi.py
wsgi.py
```

Output

```
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
Django version 5.1, using settings 'MyWebsite.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Opening http://127.0.0.1:8000/ in the browser shows the Django rocket page with the message **The install worked successfully! Congratulations**

Experiment 8

Aim: Create a Django app named blog and register it in the project's settings.

Command

python manage.py startapp blog

MyWebsite/settings.py

```
INSTALLED_APPS = [
'django.contrib.admin',
'django.contrib.auth',
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',
'blog', # newly added app
]
```

Output

```
blog/
__init__.py
admin.py
apps.py
migrations/
models.py
tests.py
views.py
$ python manage.py check
System check identified no issues (0 silenced).
```

Experiment 9

Aim: Create a view that returns a simple "Hello, Django!" response and map it to a URL.

blog/views.py

```
from django.http import HttpResponse
def hello(request):
return HttpResponse('Hello, Django!')
```

blog/urls.py (create this file)

```
from django.urls import path
from . import views
urlpatterns = [
path('hello/', views.hello, name='hello'),
]
```

MyWebsite/urls.py

```
from django.contrib import admin
from django.urls import path, include
urlpatterns = [
path('admin/', admin.site.urls),
path('', include('blog.urls')),
]
```

Output (http://127.0.0.1:8000/hello/)



Experiment 10

Aim: Create an HTML page using Django templates that displays your name and profession.

blog/views.py (add)

```
from django.shortcuts import render
def profile(request):
context = {
'name': 'Yash Bhoi',
'profession': 'Software Engineer & Web/App Developer',
}
return render(request, 'blog/profile.html', context)
```

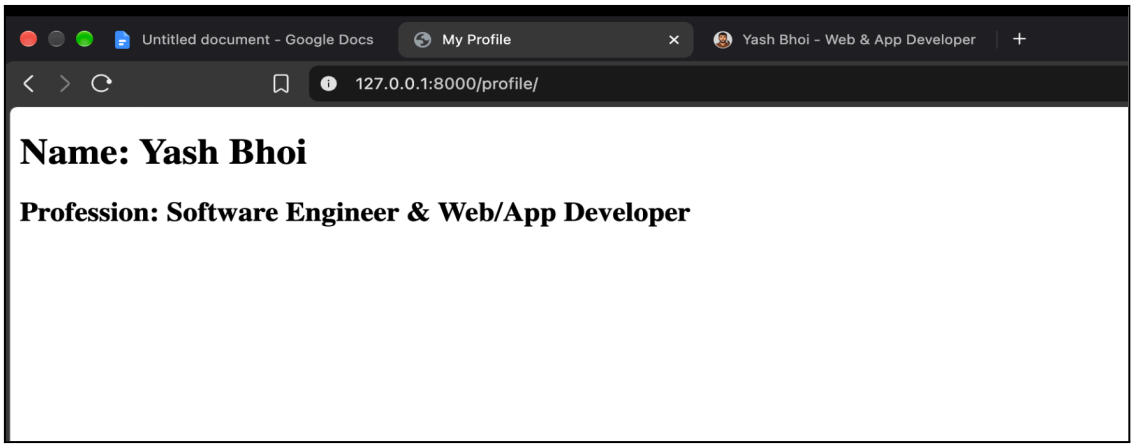
blog/urls.py (add)

```
path('profile/', views.profile, name='profile'),
```

blog/templates/blog/profile.html

```
<!DOCTYPE html>
<html>
<head>
<title>My Profile</title>
</head>
<body>
<h1>Name: {{ name }}</h1>
<h2>Profession: {{ profession }}</h2>
</body>
</html>
```

Output (http://127.0.0.1:8000/profile/)



Experiment 11

Aim: Create a base template and extend it with two different pages: home.html and about.html.

blog/templates/blog/base.html

```
<!DOCTYPE html>
<html>
<head>
<title>{% block title %}My Website{% endblock %}</title>
</head>
<body>
<nav>
<a href='/'>Home</a> | <a href='/about/'>About</a>
</nav>
<hr>
{% block content %}{% endblock %}
<hr>
<footer>© 2026 My Website</footer>
</body>
</html>
```

blog/templates/blog/home.html

```
{% extends 'blog/base.html' %}
{% block title %}Home{% endblock %}
{% block content %}
<h1>Welcome to My Website</h1>
<p>This is the home page.</p>
{% endblock %}
```

blog/templates/blog/about.html

```
{% extends 'blog/base.html' %}
{% block title %}About{% endblock %}
{% block content %}
<h1>About Us</h1>
<p>I am Yash Bhoi, a software engineer and web & app developer, and founder of YsquareTech.</p>
{% endblock %}
```

blog/views.py (add)

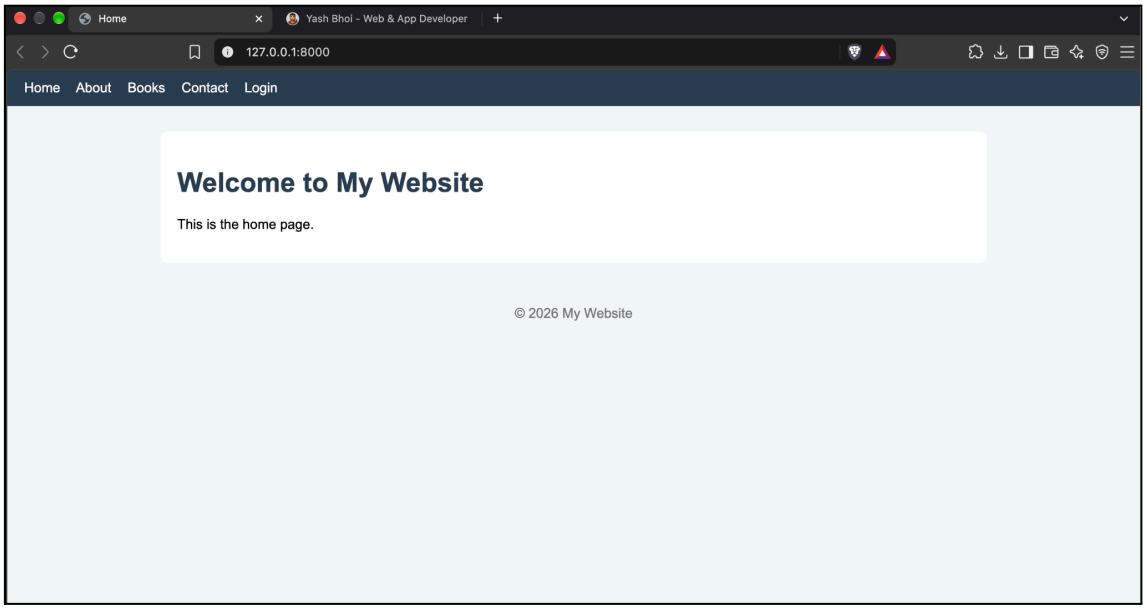
```
def home(request):
return render(request, 'blog/home.html')
def about(request):
return render(request, 'blog/about.html')
```

blog/urls.py (add)

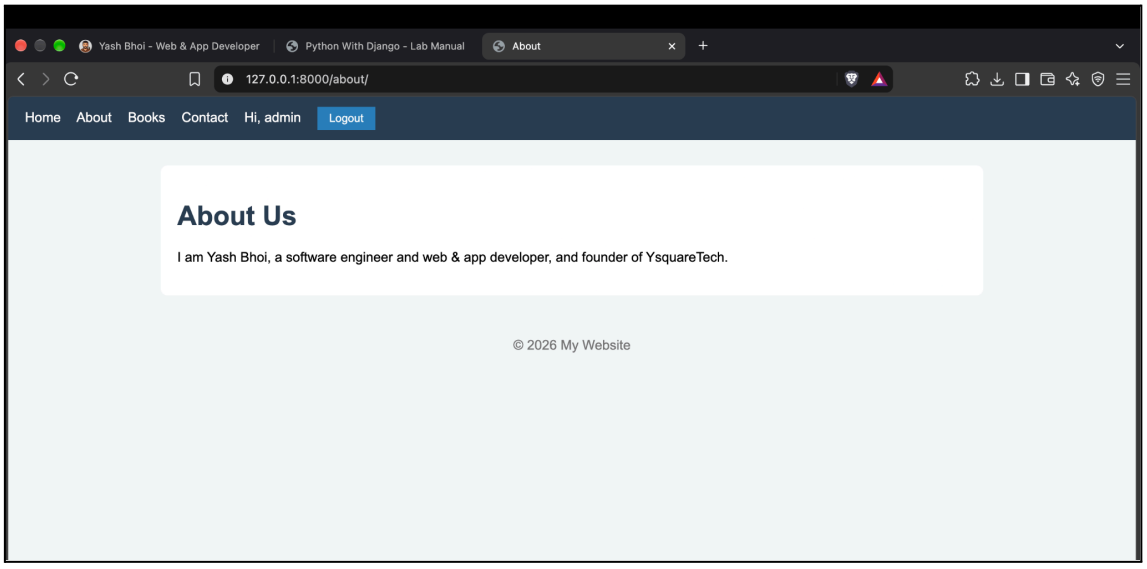
```
path('', views.home, name='home'),
path('about/', views.about, name='about'),
```

Output

Home page:



About page:



Experiment 12

Aim: Define a Book model with fields title, author and published_date, then run the migrations.

blog/models.py

```
from django.db import models
class Book(models.Model):
title = models.CharField(max_length=200)
author = models.CharField(max_length=100)
published_date = models.DateField()
def __str__(self):
return self.title
```

Commands

```
python manage.py makemigrations
python manage.py migrate
```

Output

```
Migrations for 'blog':
blog/migrations/0001_initial.py
+ Create model Book
Operations to perform:
Apply all migrations: admin, auth, blog, contenttypes, sessions
Running migrations:
Applying contenttypes.0001_initial... OK
Applying auth.0001_initial... OK
Applying admin.0001_initial... OK
Applying blog.0001_initial... OK
Applying sessions.0001_initial... OK
```

Experiment 13

Aim: Add books via the admin panel and display their titles using the Django ORM; create a form where users submit their name and email and show the submitted values on the next page.

Part A: Admin and ORM

blog/admin.py

```
from django.contrib import admin
from .models import Book
admin.site.register(Book)
```

Commands

```
python manage.py createsuperuser
python manage.py runserver
```

Log in at /admin/ and add three books: Python Crash Course, Django for Beginners, Clean Code.

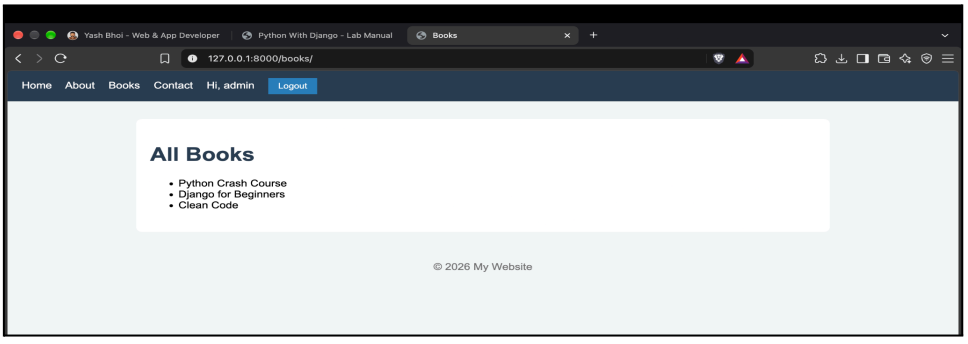
blog/views.py (add)

```
from .models import Book
def book_list(request):
books = Book.objects.all()
return render(request, 'blog/book_list.html', {'books': books})
blog/templates/blog/book_list.html
{% extends 'blog/base.html' %}
{% block content %}
<h1>All Books</h1>
<ul>
{% for book in books %}
<li>{{ book.title }}</li>
{% empty %}
<li>No books found.</li>
{% endfor %}
</ul>
{% endblock %}
```

blog/urls.py (add)

```
path('books/', views.book_list, name='book_list'),
```

Output (http://127.0.0.1:8000/books/)



Part B: Name and email form

blog/forms.py

```
from django import forms
class ContactForm(forms.Form):
name = forms.CharField(max_length=100)
email = forms.EmailField()
```

blog/views.py (add)

```
from .forms import ContactForm
def contact(request):
if request.method == 'POST':
if form.is_valid():
return render(request, 'blog/result.html', form.cleaned_data)
else:
form = ContactForm()
return render(request, 'blog/form.html', {'form': form})
```

blog/templates/blog/form.html

```
{% extends 'blog/base.html' %}
{% block content %}
<h1>Contact Form</h1>
<form method='post'>
{% csrf_token %}
{{ form.as_p }}
<button type='submit'>Submit</button>
</form>
{% endblock %}
```

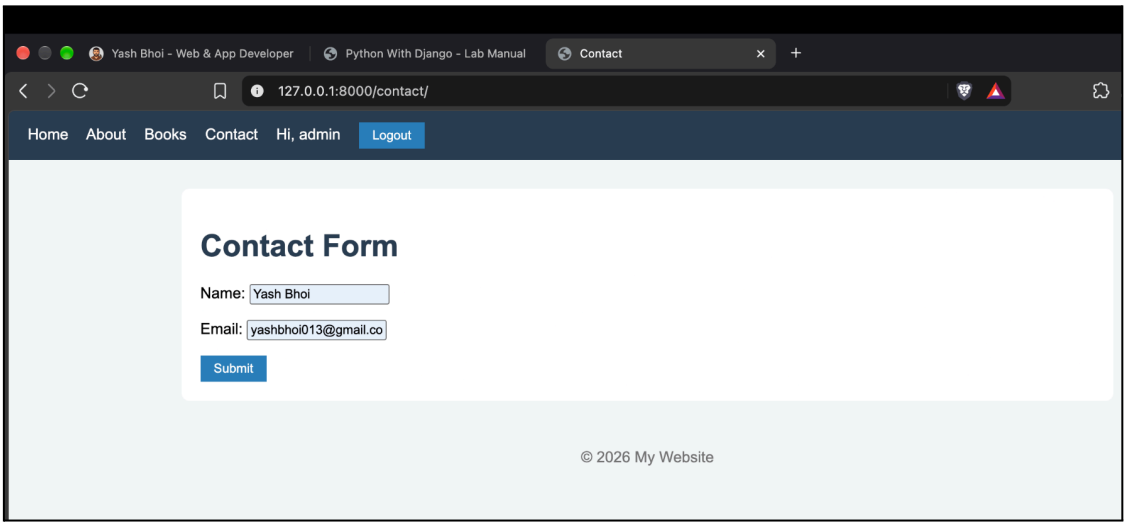
blog/templates/blog/result.html

```
{% extends 'blog/base.html' %}
{% block content %}
<h1>Submitted Details</h1>
<p>Name: {{ name }}</p>
<p>Email: {{ email }}</p>
{% endblock %}
```

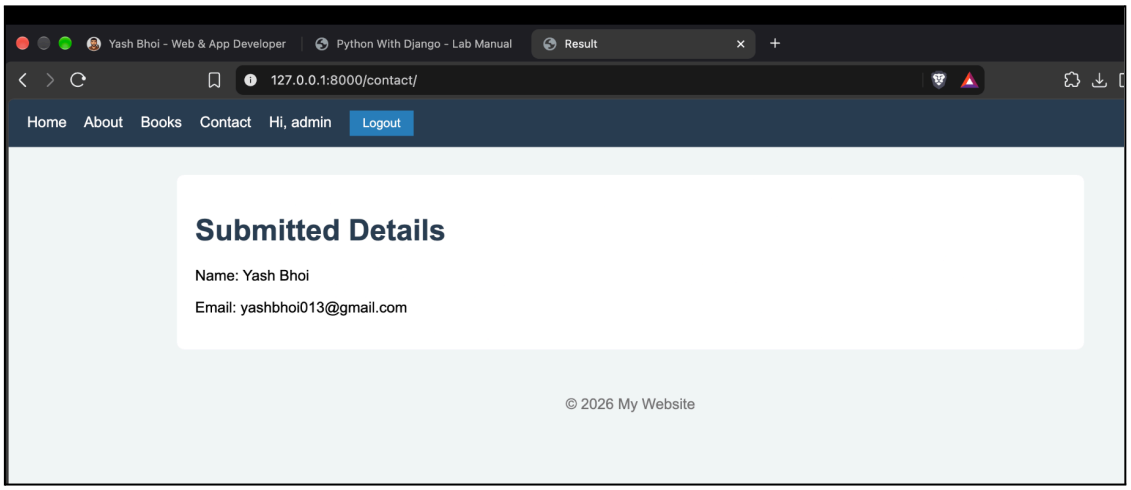
blog/urls.py (add)

```
path('contact/', views.contact, name='contact'),
```

Output



After submit:



Experiment 14

Aim: Add login/logout using Django's built-in auth system and style the site with a CSS file linked through the{% static %} tag.

MyWebsite/urls.py

```
urlpatterns = [
path('admin/', admin.site.urls),
path('accounts/', include('django.contrib.auth.urls')),
path('', include('blog.urls')),
]
```

MyWebsite/settings.py (add at the bottom)

```
LOGIN_REDIRECT_URL = '/'
LOGOUT_REDIRECT_URL = '/accounts/login/'
STATIC_URL = 'static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
```

templates/registration/login.html (templates folder in the project root, or blog/templates/registration/)

```
{% extends 'blog/base.html' %}
{% block content %}
<h1>Login</h1>
<form method='post'>
{% csrf_token %}
{{ form.as_p }}
<button type='submit'>Login</button>
</form>
{% endblock %}
```

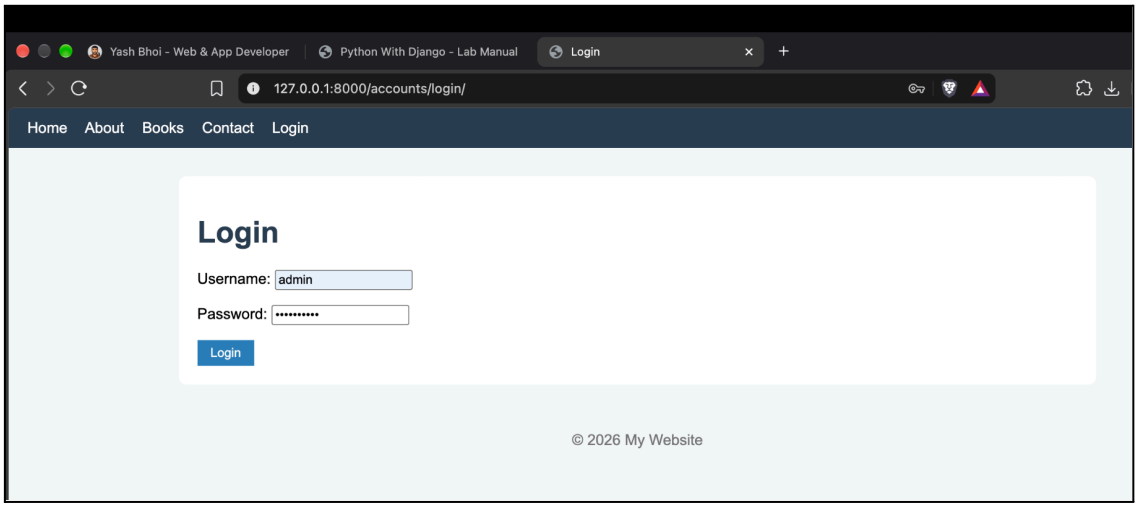
blog/templates/blog/base.html (updated)

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
<title>{% block title %}My Website{% endblock %}</title>
<link rel='stylesheet' href="{% static 'css/style.css' %}">
</head>
<body>
<nav>
<a href='/'>Home</a>
<a href='/about/'>About</a>
{% if user.is_authenticated %}
<span>Hi, {{ user.username }}</span>
<form method='post' action="{% url 'logout' %}" style='display:inline'>
{% csrf_token %}
<button type='submit'>Logout</button>
</form>
{% else %}
<a href="{% url 'login' %}">Login</a>
{% endif %}
</nav>
<div class='container'>
{% block content %}{% endblock %}
</div>
</body>
</html>
```

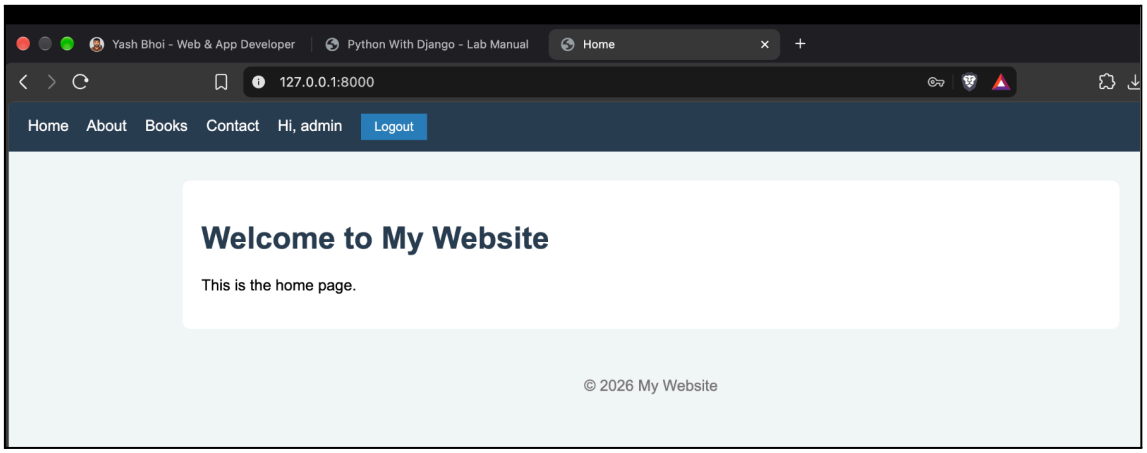
static/css/style.css

```
body { font-family: Arial, sans-serif; background: #f4f6f9; margin: 0; }
nav { background: #2c3e50; padding: 12px 20px; }
nav a, nav span { color: white; margin-right: 15px; text-decoration: none; }
.container {
width: 70%; margin: 30px auto; background: white;
padding: 20px; border-radius: 8px;
}
h1 { color: #2c3e50; }
button {
background: #2980b9; color: white; border: none;
padding: 6px 14px; cursor: pointer;
}
```

Output



After login:



Experiment 15

Aim: Build a REST API using Django REST Framework to perform CRUD operations on a Product model.

Installation

pip install djangorestframework
MyWebsite/settings.py
INSTALLED_APPS = [
...existing apps...
'rest_framework',
'blog',
]

blog/models.py (add)

class Product(models.Model):
name = models.CharField(max_length=100)
price = models.DecimalField(max_digits=10, decimal_places=2)
quantity = models.PositiveIntegerField(default=0)
def __str__(self):
return self.name

Commands

python manage.py makemigrations
python manage.py migrate
blog/serializers.py
from rest_framework import serializers
from .models import Product
class ProductSerializer(serializers.ModelSerializer):
class Meta:
model = Product
fields = ['id', 'name', 'price', 'quantity']

blog/views.py (add)

from rest_framework import viewsets
from .models import Product
from .serializers import ProductSerializer
class ProductViewSet(viewsets.ModelViewSet):
queryset = Product.objects.all()
serializer_class = ProductSerializer

blog/urls.py (add)

from rest_framework.routers import DefaultRouter
router = DefaultRouter()
router.register('products', views.ProductViewSet)
urlpatterns += router.urls

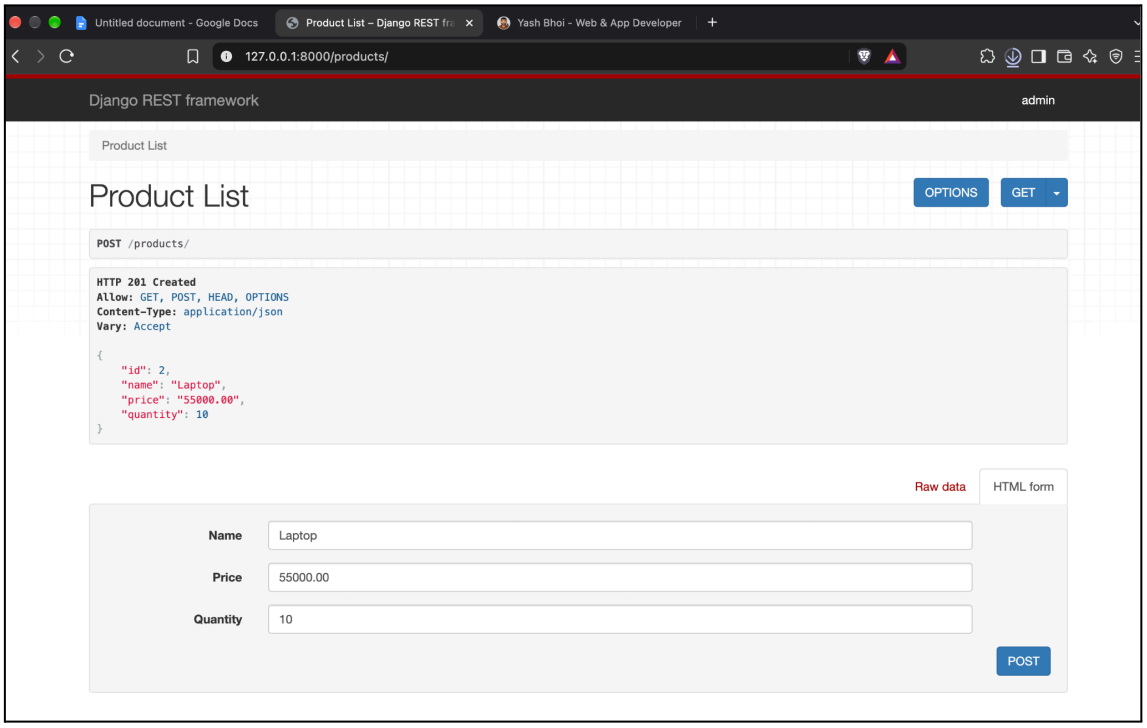
Output (tested with curl)

CREATE
$ curl -X POST http://127.0.0.1:8000/products/ \
-H "Content-Type: application/json" \
-d '{"name":"Laptop","price":"55000.00","quantity":10}'
HTTP 201 Created
{"id":7,"name":"Laptop","price":"55000.00","quantity":10}
READ (list)
$ curl http://127.0.0.1:8000/products/
HTTP 200 OK
[{"id":7,"name":"Laptop","price":"55000.00","quantity":10}]

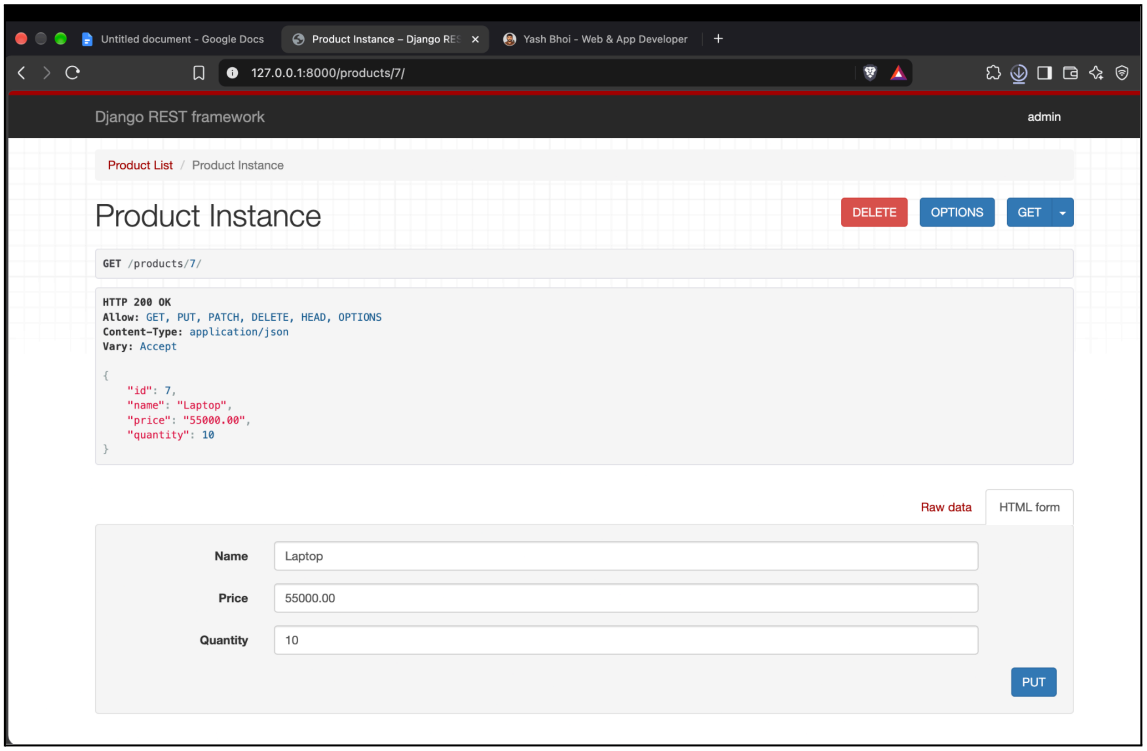
```
# READ (single)
$ curl http://127.0.0.1:8000/products/7/
HTTP 200 OK
{"id":7,"name":"Laptop","price":"55000.00","quantity":10}
# UPDATE
$ curl -X PUT http://127.0.0.1:8000/products/1/ \
-H "Content-Type: application/json" \
-d '{"name":"Laptop","price":"52000.00","quantity":8}'
HTTP 200 OK
{"id":7,"name":"Laptop","price":"52000.00","quantity":8}
# DELETE
$ curl -X DELETE http://127.0.0.1:8000/products/1/
HTTP 204 No Content
```

The same operations can also be performed from the browsable API at http://127.0.0.1:8000/products/.

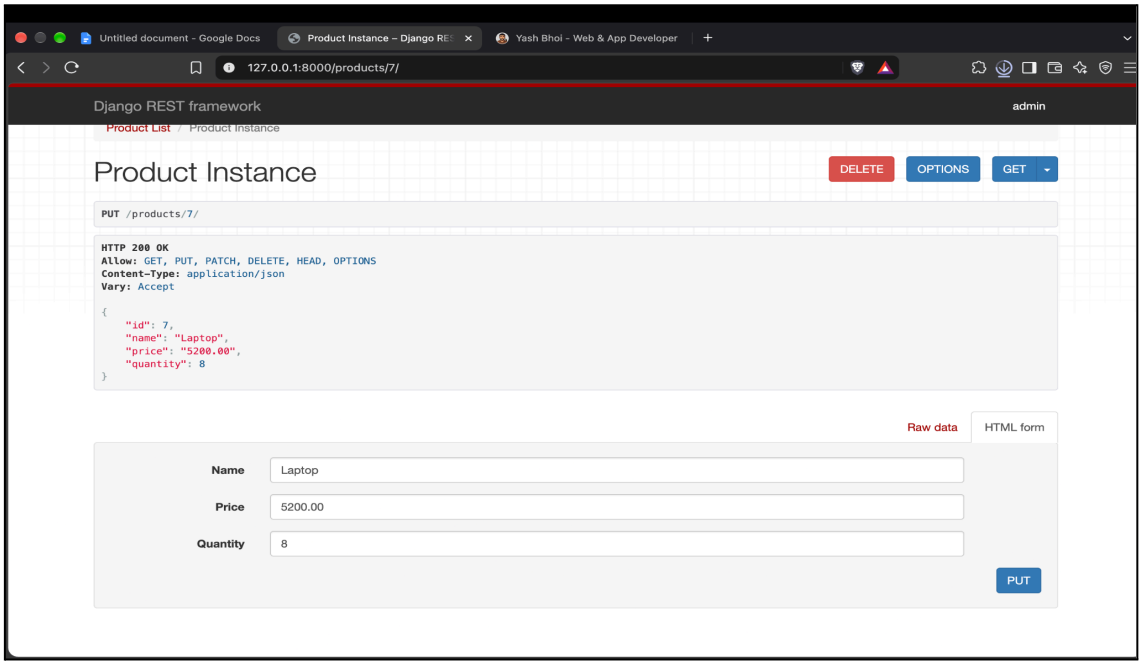
Add Product:



View Product using id:



Update Product



After deleting :

